

实验1:无向图的深度优先遍历（DFS）

实现原理

1. 邻接矩阵存储:

使用一个 $n \times n$ 的二维数组 `graph[MAXN][MAXN]` 存储图的结构。`graph[i][j] == 1` 表示从顶点 `i` 到顶点 `j` 存在一条边，`0` 表示无边。

2. DFS:

从某个初始顶点 `v` 出发，首先访问该顶点，然后依次从 `v` 的未被访问的邻接点出发进行深度优先遍历，直到图中所有和 `v` 有路径相通的顶点都被访问到。这是一个递归的过程。其中，`visited[MAXN]` 数组用于标记顶点是否被访问过，`order[MAXN]` 数组用于记录访问顶点的顺序。

本实验定义的核心数据结构与变量如下：

```
#define MAXN 50
int graph[MAXN][MAXN]; // 邻接矩阵
bool visited[MAXN];    // 访问标记数组
int order[MAXN];       // 记录顶点的访问次序
int n, idx = 0;        // n为顶点数，idx为访问次序索引
```

核心算法实现：

```
void dfs(int v) {
    visited[v] = true;    // 1. 标记当前顶点已访问
    order[idx] = v;      // 2. 将当前顶点存入遍历结果数组
    idx++;

    // 3. 搜索v的所有邻接点
    for(int i = 0; i < n; i++) {
        // 如果存在边且邻接点未被访问
        if(graph[v][i] == 1 && visited[i] == false) {
            dfs(i);      // 递归访问邻接点
        }
    }
}

int main() {
    //读取数据部分代码省略
    for(int i=0;i<n;i++){
        if(visited[i]==false){
            dfs(i);
        }
    }
    for(int i=0;i<n;i++){
        printf("%d ", order[i]);
    }
    return 0;
}
```

实验2：无向图的广度优先遍历（BFS）

实现原理

BFS 算法必须借助队列数据结构来实现，传统 BFS 需要两个独立的数组：

```
int queue[MAXN];    // 队列，管理待处理顶点
int result[MAXN];   // 结果数组，记录访问顺序
```

本次代码实现将两者合二为一，用一个order数组实现：

```
int order[MAXN];    // 前半部分是已处理的（结果），后半部分是待处理的（队列）
int idx = 0;        // 队尾指针，指向下一个入队位置
int id = 0;         // 队首指针，指向下一个待出队元素
```

操作	代码实现	含义
入队	order[idx++] = j	在队尾插入，idx 后移
出队	v = order[id++]	从队首取出，id 后移
判空	id < idx	队首未追上队尾 → 非空
判满	idx == MAXN	队尾到达数组末尾（本实验不考虑溢出）

这种设计是可行的关键在于 BFS 的访问顺序恰好就是入队顺序：顶点被发现的顺序严格等于入队的顺，id 指针永远不会超过 idx 指针，当 id 追上 idx 时，所有发现的顶点都已处理完毕。处理完毕时，order 正好按 BFS 顺序装满

核心算法实现与注释：

```
#include <stdio.h>
#include <stdbool.h>

#define MAXN 50

int graph[MAXN][MAXN];    // 邻接矩阵
bool visited[MAXN];       // 访问标记
int order[MAXN];          // 队列 + 结果（双重身份）
int n, idx = 0, id = 0;   // n=顶点数，idx=队尾，id=队首

int main() {
    scanf("%d", &n);
```

```

// 读取邻接矩阵
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%1d", &graph[i][j]);
    }
    visited[i] = false;
}

// 处理非连通图：外层循环确保每个连通分量都被访问
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        // === 初始化新连通分量 ===
        order[idx++] = i; // 起始顶点入队（队尾指针后移）
        visited[i] = true; // 标记已访问

        // === BFS 核心循环 ===
        while (id < idx) { // 队列非空时继续
            int v = order[id++]; // 出队：取出队首，队首指针后移

            // 扫描 v 的所有邻接点
            for (int j = 0; j < n; j++) {
                if (graph[v][j] == 1 && !visited[j]) {
                    visited[j] = true; // 标记已访问
                    order[idx++] = j; // 入队：追加到队尾
                }
            }
        }
    }
}

// 输出 BFS 遍历顺序
for (int i = 0; i < n; i++) {
    printf("%d ", order[i]);
}
printf("\n");

return 0;
}

```

实验3：基于A*算法的八数码问题求解

实现原理：

A*算法维护两个值：

$g(n)$ ：从初始状态到当前状态 n 的实际代价

$h(n)$ ：从当前状态 n 到目标状态的估计代价（启发函数值）

$f(n) = g(n) + h(n)$ ：评估函数，表示经过 n 的总体估计代价

A*算法总是优先扩展 $f(n)$ 值最小的节点，这保证了找到的第一个解就是最优解（当 $h(n)$ 是可采纳的时），且比盲

目搜索访问更少的状态

本实验使用曼哈顿距离作为启发函数：

$$h(n) = \sum | \text{当前数字所在行} - \text{目标位置行} | + | \text{当前数字所在列} - \text{目标位置列} |$$

状态编码与哈希映射：

八数码的状态是一个3×3矩阵，但在程序中需要高效存储和比较，因此将3×3矩阵压缩为一个整数

```
state = [2, 8, 3, 1, 6, 4, 7, 0, 5]
code = 283164705
```

g_map 的作用：

```
unordered_map<int, int> g_map; // 状态码 -> 到达该状态的最小步数
```

用于避免重复访问：如果一个状态已经有更短的到达路径，则跳过。

核心代码实现原理：

1，可解性判断

八数码问题并非所有初始状态都能到达目标状态。判断依据是逆序数奇偶性：

```
int inversion(const vector<int>& state) {
    int inv = 0;
    for (int i = 0; i < 9; ++i) {
        if (state[i] == 0) continue; // 忽略空格
        for (int j = i + 1; j < 9; ++j) {
            if (state[j] == 0) continue;
            if (state[i] > state[j]) inv++; // 计算逆序对
        }
    }
    return inv % 2; // 返回奇偶性
}
```

由于0视为空格不计入逆序数，因此若空格左右移动，则逆序数不变，若上下移动，相当于做了两次对换，逆序数+2或-2，因此奇偶性不变，故初始状态和目标状态的逆序数奇偶性不同则无解。

2, A*搜索核心过程

```
int aStar(const vector<int>& start) {
    // 1. 可解性判断
    if (inversion(start) != inversion(target)) {
        return -1;
    }

    // 2. 方向数组 (上、下、左、右)
    const int dx[4] = {-1, 1, 0, 0};
    const int dy[4] = {0, 0, -1, 1};

    // 3. 优先队列: 存储 (f, g, code, zero_pos)
    //     greater<Node> 保证 f 最小的在队首
    using Node = tuple<int, int, int, int>;
    priority_queue<Node, vector<Node>, greater<Node>> pq;

    unordered_map<int, int> g_map; // 记录到达各状态的最小 g 值

    // 4. 初始化起点
    int start_code = encode(start);
    int start_zero = find(start.begin(), start.end(), 0) - start.begin();
    int start_h = manhattan(start);
    pq.push({start_h, 0, start_code, start_zero});
    g_map[start_code] = 0;

    // 5. 搜索主循环
    while (!pq.empty()) {
        auto [f, g, code, zero] = pq.top();
        pq.pop();

        // 惰性删除: 跳过不是最优的节点
        if (g_map[code] != g) continue;

        // 解码: 从整数恢复为向量
        vector<int> state(9);
        int tmp = code;
        for (int i = 8; i >= 0; --i) {
            state[i] = tmp % 10;
            tmp /= 10;
        }

        // 到达目标状态
        if (state == target) {
            return g;
        }

        // 6. 扩展邻居节点
        int x = zero / 3, y = zero % 3;
        for (int k = 0; k < 4; ++k) {
            int nx = x + dx[k], ny = y + dy[k];
```

```

        if (nx < 0 || nx >= 3 || ny < 0 || ny >= 3) continue;

        int nz = nx * 3 + ny; // 新空格位置
        vector<int> new_state = state;
        swap(new_state[zero], new_state[nz]); // 移动空格

        int new_code = encode(new_state);
        int new_g = g + 1;

        // 如果是更优路径，更新
        auto it = g_map.find(new_code);
        if (it == g_map.end() || new_g < it->second) {
            g_map[new_code] = new_g;
            int new_h = manhattan(new_state);
            pq.push({new_g + new_h, new_g, new_code, nz});
        }
    }
}

return -1; // 无解
}

```

以下为部分代码的详细解释：

2.1 移动方向的预定义

```

// 方向：上，下，左，右
const int dx[4] = {-1, 1, 0, 0};
const int dy[4] = {0, 0, -1, 1};

```

定义空格（0）的四种可能移动方向。

空间坐标与一维索引的对应关系如下：

一维索引：	对应的二维坐标：
0 1 2	(0,0) (0,1) (0,2)
3 4 5	(1,0) (1,1) (1,2)
6 7 8	(2,0) (2,1) (2,2)

转换公式：

二维到一维：index = x * 3 + y

一维到二维：x = index / 3, y = index % 3

因此可以用方向数组来表示移动：

dx[0]=-1, dy[0]=0	→ 向上移动（行号-1）
dx[1]=1, dy[1]=0	→ 向下移动（行号+1）
dx[2]=0, dy[2]=-1	→ 向左移动（列号-1）
dx[3]=0, dy[3]=1	→ 向右移动（列号+1）

2.2 核心数据结构定义

```
// 定义优先队列节点的类型
// 节点包含：(f值, g值, 状态编码, 空格位置)
using Node = tuple<int, int, int, int>;

// 优先队列，按 f 值从小到大排序（最小堆）
priority_queue<Node, vector<Node>, greater<Node>> pq;

// 哈希表：记录到达每个状态的最小实际代价 g
// key: 状态编码 (int)
// value: 从起点到该状态的最小步数 g
unordered_map<int, int> g_map;
```

2.3 初始化搜索起点

```
// 将初始状态编码为整数
int start_code = encode(start);

// 找到空格（0）在一维数组中的位置
int start_zero = find(start.begin(), start.end(), 0) - start.begin();

// 计算初始状态的启发函数值（曼哈顿距离）
int start_h = manhattan(start);

// 将起点加入优先队列
// f = start_h (因为 g=0, 所以 f = 0 + h = h)
// g = 0, code = start_code, zero = start_zero
pq.push({start_h, 0, start_code, start_zero});

// 记录到达起点的 g 值为 0
g_map[start_code] = 0;
```

2.4 取出最有希望的节点

```
// 从优先队列中取出 f 值最小的节点
auto [f, g, code, zero] = pq.top();
pq.pop();
```

这是A*算法的核心操作，用 `pq.top()` 返回优先队列的首元素，即当前 $f=g+h$ 最小的节点，随后将元组的四个元素解包：

f = 该节点的评估值 $g+h$

g = 从起点到达该状态的实际步数

$code$ = 该状态的整数编码

$zero$ = 空格在该状态中的位置（0~8）

`pq.pop()` 将其从队列中移除

2.5 删除非最优情况

```
// 如果当前g值不是最优，跳过
if (g_map[code] != g) continue;
```

由于发现更优路径时会重新将状态入队，而非更新优先队列中的旧节点，所以同一状态可能在优先队列中出现多次：每次路径更新时，都会将旧状态重新入队，并更新g值。因此，在取出节点时，我们需要检查其g值是否为最优，如果不是，则跳过该节点，通过这种方式减少大量不必要的重复搜索。

2.6 扩展后继状态

```
// 将空格的一维位置转换为二维坐标
int x = zero / 3, y = zero % 3;

// 尝试所有四个方向
for (int k = 0; k < 4; ++k) {
    int nx = x + dx[k], ny = y + dy[k]; // 新坐标

    // 边界检查：确保新坐标在棋盘内
    if (nx < 0 || nx >= 3 || ny < 0 || ny >= 3) continue;

    // 将新坐标转回一维位置
    int nz = nx * 3 + ny;

    // 复制当前状态
    vector<int> new_state = state;

    // 交换空格与新位置的内容（模拟移动）
    swap(new_state[zero], new_state[nz]);

    // 将新状态编码为整数
    int new_code = encode(new_state);

    // 新状态的 g 值 = 当前 g 值 + 1
    int new_g = g + 1;
```

2.7 判断是否加入队列

```
// 在 g_map 中查找新状态
auto it = g_map.find(new_code);

// 条件1：第一次发现该状态 (it == g_map.end())
// 条件2：发现了更短的到达路径 (new_g < it->second)
if (it == g_map.end() || new_g < it->second) {
```


2.8 将新状态加入优先队列

```
// 更新（或创建）到达该状态的最优 g 值
g_map[new_code] = new_g;

// 计算新状态的启发函数值
int new_h = manhattan(new_state);

// 将新节点加入优先队列
// f = new_g + new_h
// g = new_g
// code = new_code
// zero = nz（新空格位置）
pq.push({new_g + new_h, new_g, new_code, nz});
    }
}
}
```

详细解析：

1. `g_map[new_code] = new_g`
 - 记录到达 `new_code` 状态的最少步数
 - 如果之前已存在，则覆盖为更小的值
2. `new_h = manhattan(new_state)`
 - 计算新状态到目标的曼哈顿距离
 - 这就是 A* 的启发函数 $h(n)$
3. `pq.push({new_g + new_h, new_g, new_code, nz})`
 - 计算 $f=g+h$ 作为评估值
 - 将所有信息打包成节点，加入优先队列

至此完成整个A*算法函数的设计，只需根据实验要求在主函数中读入数据，调用该函数，有解时输出步数，无解时输出-1再在主函数中输出"Impossible"即可。

实验4： α - β 剪枝

1，算法原理

1.1 Minimax 算法

Minimax 是解决零和博弈的经典算法，其核心思想是：

Alice（最大化玩家）：在所有可选操作中选择能带来最大评估值的操作

Bob（最小化玩家）：在所有可选操作中选择能带来最小评估值的操作

双方都假设对手会采取最优策略，通过递归构建博弈树，从叶子节点向上传递最优值。

1.2 Alpha-Beta 剪枝

Alpha-Beta剪枝是Minimax的优化版本，通过维护两个边界值来避免探索不可能影响最终结果的分支。

参数	含义	初始化
α	MAX玩家能够保证的当前最优下界	$-\infty$
β	MIN玩家能够保证的当前最优上界	$+\infty$

剪枝条件：

MAX层：当发现某个节点的值 $\geq \beta$ 时，父节点（MIN层）不会选择这条路径，则剪枝

MIN层：当发现某个节点的值 $\leq \alpha$ 时，父节点（MAX层）不会选择这条路径，则剪枝

1.3 累积得分传递机制

本实验的一个关键设计是将累积得分作为参数传递，而非在叶子节点重新计算。

```
int minimax(Board& board, int depth, bool is_alice,
            int alpha, int beta, int current_score)
```

`current_score` 是到达当前状态时已经累积的总分，每一步执行操作后，将 `current_score + 本次得分` 传给下一层，到达叶子节点（`depth == 0`）时，直接返回 `current_score`

优点：

- 1. 避免在叶子节点重复遍历路径计算总分
- 2. 递归清晰，每层只需关注当前操作
- 3. 适配Minimax框架

2，核心代码实现

2.1 棋盘结构体

```
struct Board {
    int grid[4][4];
    // ...
};
```

使用4*4二维数组*储棋盘状态，直接操作数组便于旋转和回溯。

2.2 旋转操作

逆时针旋转

```
int rotate_ccw(int r, int c) {
    int a = grid[r][c];      // 左上
    int b = grid[r][c+1];    // 右上
    int c_val = grid[r+1][c]; // 左下
    int d = grid[r+1][c+1];  // 右下

    // 逆时针旋转：左上→右上，右上→右下，右下→左下，左下→左上
    grid[r][c] = b;          // 左上 ← 右上
    grid[r][c+1] = d;        // 右上 ← 右下
    grid[r+1][c] = a;        // 左下 ← 左上
    grid[r+1][c+1] = c_val;  // 右下 ← 左下

    return a + b + c_val + d; // 返回得分
}
```

顺时针旋转

```
void rotate_cw(int r, int c) {
    int a = grid[r][c];
    int b = grid[r][c+1];
    int c_val = grid[r+1][c];
    int d = grid[r+1][c+1];

    grid[r][c] = c_val;  // 左上 ← 左下
    grid[r][c+1] = a;    // 右上 ← 左上
    grid[r+1][c] = d;    // 左下 ← 右下
    grid[r+1][c+1] = b;  // 右下 ← 右上
}
```

3.3 Minimax + Alpha-Beta 搜索函数

完整递归函数如下

```
int minimax(Board& board, int depth, bool is_alice,
            int alpha, int beta, int current_score) {
    // 递归终止条件
    if (depth == 0) {
        return current_score;
    }
}
```

```

if (is_alice) {
    // ===== Alice层: 最大化 =====
    int max_eval = -1e9;

    // 枚举所有 3x3 = 9 种可能的操作
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            // 1. 执行操作, 获得本次得分
            int score = board.rotate_ccw(r, c);

            // 2. 递归: 传给Bob层, 累加得分
            int eval = minimax(board, depth - 1, false,
                               alpha, beta, current_score + score);

            // 3. 回溯: 恢复棋盘
            board.rotate_cw(r, c);

            // 4. 更新最大值和alpha
            max_eval = max(max_eval, eval);
            alpha = max(alpha, eval);

            // 5. Alpha-Beta剪枝
            if (beta <= alpha) break;
        }
    }
    return max_eval;
} else {
    // ===== Bob层: 最小化 =====
    int min_eval = 1e9;

    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            int score = board.rotate_ccw(r, c);
            int eval = minimax(board, depth - 1, true,
                               alpha, beta, current_score + score);
            board.rotate_cw(r, c);

            min_eval = min(min_eval, eval);
            beta = min(beta, eval);

            // Alpha-Beta剪枝
            if (beta <= alpha) break;
        }
    }
    return min_eval;
}
}

```

以下为详解：

3.3.1 参数详解

参数	类型	含义	传递方向
board	Board&	棋盘状态引用，全程操作同一棋盘	引用传递，共享状态
depth	int	剩余搜索深度（还剩几步未走）	自顶向下递减
is_alice	bool	当前层是Alice（true）还是Bob（false）	自顶向下交替
alpha	int	MAX方能保证的当前最优下界	自顶向下传递约束
beta	int	MIN方能保证的当前最优上界	自顶向下传递约束
current_score	int	到达当前状态已累积的总得分	自顶向下累加

返回值：在当前棋盘状态下，剩余depth步操作后，双方均采用最优策略时的最终总得分。

3.3.2 递归终止条件

```
// 递归边界：所有回合结束，直接返回全局累积总分
if (depth == 0) {
    return current_score;
}
```

3.3.3 Alice层（最大化玩家）

```
if (is_alice) {
    int max_eval = -1e9; // Alice 目标是最大化总分
```

初始化局部变量：max_eval 初始化为极小值-1e9，近似负无穷，确保第一个子节点的评估值一定会大于初始值，从而被正确更新
随后遍历所有 9 种可能的操作，执行操作并获取分数

```
for (int r = 0; r < 3; ++r) {
    for (int c = 0; c < 3; ++c) {
        int score = board.rotate_ccw(r, c); // 执行操作，获取本次得分
        // 将本次得分累加到 current_score 往下传，返回的是绝对总分
        int eval = minimax(board, depth - 1, false, alpha, beta, current_score + score);
```

上面代码中的最后一行为递归操作，这一操作做了四件事：

- 1. depth - 1：剩余步数减1
- 2. false：下一层轮到 Bob
- 3. alpha, beta：传递当前的剪枝边界（自顶向下）
- 4. current_score + score：累加本次得分

```
board.rotate_cw(r, c); // 状态回溯
```

这行代码将棋盘恢复到本次操作执行前的状态，以便进行遍历9种操作的下一分支(逆时针旋转通过顺时针旋转函数恢复)。

随后更新最优值和Alpha：

```
max_eval = max(max_eval, eval);
alpha = max(alpha, eval);
```

两行代码分别更新两个不同的值：

变量	作用域	含义
max_eval	当前层局部	当前Alice层所有子节点中的最大值
alpha	跨层级全局	MAX方能保证的全局最优下界

Alpha-Beta 剪枝

```
// Alpha-Beta 剪枝
if (beta <= alpha) {
    break;
}
```

剪枝条件含义：alpha是当前MAX方能保证的全局最优下界，即“当前MAX节点最少能拿到alpha分”，beta是当前MIN方能保证的全局最优上界，即“当前MIN节点能保证最终总分不超过beta”。如果beta <= alpha，则说明当前MAX节点能提供的分数（记为score）必定满足score>=alpha>=beta，因此无法让父MIN节点选择自己，可以剪枝。

返回当前Alice层能获得的最大总分。

```
return max_eval;
```

3.3.4 Bob层（最小化玩家）

```
    } else {
        int min_eval = 1e9; // Bob 目标是最小化总分
```

初始化局部变量：

- min_eval 初始化为极大值1e9，近似正无穷
- 确保第一个子节点的评估值一定会小于初始值

枚举操作、递归、回溯

```
for (int r = 0; r < 3; ++r) {
    for (int c = 0; c < 3; ++c) {
        int score = board.rotate_ccw(r, c);
        int eval = minimax(board, depth - 1, true, alpha, beta, current_score + score);
        board.rotate_cw(r, c);
    }
}
```

与Alice层完全相同的步骤：

- 1. 枚举9种操作
- 2. 执行旋转，获取得分
- 3. 递归调用（true 表示下一层是Alice）
- 4. 累加得分：current_score + score
- 5. 回溯恢复棋盘

更新最优值和Beta

```
min_eval = min(min_eval, eval);
beta = min(beta, eval);
```

变量	更新方式	含义
min_eval	min(min_eval, eval)	当前Bob层所有子节点中的最小值
beta	min(beta, eval)	MIN方能保证的全局最优上界

Alpha-Beta剪枝（Bob层）

```
if (beta <= alpha) {
    break;
}
```

剪枝条件仍为 `beta <= alpha`，但含义不同，此时如果`beta <= alpha`，则说明当前MIN节点能提供的分数（记为score）必定满足`score<=beta<=alpha`，因此无法让父MAX节点选择自己，可以剪枝。

```
return min_eval;
}
```

返回当前Bob层能强制达到的最小总分。

3.4 主函数与输入输出

```
void solve() {
    int k;
    cin >> k;
    Board board;

    // 读取4x4棋盘
    for (int r = 0; r < 4; ++r) {
        for (int c = 0; c < 4; ++c) {
            cin >> board.grid[r][c];
        }
    }

    // 2k步, Alice先手, 初始累积得分=0
    // alpha=-∞, beta=+∞
    cout << minimax(board, 2 * k, true, -1e9, 1e9, 0) << "\n";
}

int main() {
    ios_base::sync_with_stdio(false); // 加速IO
    cin.tie(NULL);

    int t;
    cin >> t;
    while (t--) solve(); // 处理多个测试用例
    return 0;
}
```

至此完成整个算法设计。